

# Improving Resilient Planning Through Landmarks and Regressed State Formulas

Alberto Rovetta<sup>a</sup>, Diego Aineto<sup>a</sup>, Alfonso Emilio Gerevini<sup>a,\*</sup>, Enrico Scala<sup>a</sup> and Ivan Serina<sup>a</sup>

<sup>a</sup>Department of Information Engineering, University of Brescia, Italy

ORCID (Alberto Rovetta): <https://orcid.org/0009-0001-6224-1251>, ORCID (Diego Aineto):

<https://orcid.org/0000-0001-8173-0109>, ORCID (Alfonso Emilio Gerevini):

<https://orcid.org/0000-0001-9008-6386>, ORCID (Enrico Scala): <https://orcid.org/0000-0003-2274-875X>, ORCID (Ivan Serina): <https://orcid.org/0000-0002-7785-9492>

**Abstract.** In real-world scenarios, the successful execution of an agent's planned actions is not always guaranteed, as actions may fail in unpredictable ways that are not explicitly modeled. To address this challenge, the concept of Resilient Planning and the RESPLAN framework were introduced focusing on the generation of  $k$ -resilient plans that enable an agent to reach its goals even in the presence of up to  $k$  execution failures. In this paper, we propose a new version of the RESPLAN planning algorithm based on two significant enhancements. The first incorporates landmarks into a pruning strategy, enabling the planner to avoid unnecessary explorations and yielding substantial performance gains, especially when no resilient plan exists. The second introduces a planning adaptation strategy exploiting regressed state formulas to support the search process during (re)planning, reducing the number of iterations required when a resilient plan does exist. We compare our methods against RESPLAN and other baselines, demonstrating substantial improvements across multiple domains.

## 1 Introduction

In classical STRIPS planning [3, 5, 6], the objective is to generate a sequence of actions (a plan) whose execution starting from a given initial state generates a goal state [5, 6]. Although these plans are validated against abstract models of actions and states, real-world execution can result in unexpected failures due to unpredicted exogenous events, equipment malfunctions, or incomplete action models. A common strategy to handle such failures is to interleave plan execution with replanning when a failure occurs (e.g. [6]). However, this does not always ensure goal achievement after a failure. To address this limitation, the Resilient Planning approach and the RESPLAN algorithm aim to construct robust plans that guarantee goal achievement (by alternative plans) even in the presence of up to a bounded number of action execution failures [1]. RESPLAN computes such resilient plans by identifying bounded resilient states, that are the only states a resilient plan can traverse, and that can be leveraged as fallback points for effective replanning. The main limitations of this algorithm lie in the computational cost of proving that a state is resilient, and, more substantially, in the need to exhaustively explore all potential options to verify the lack of resilience in a given state.

In this paper, we propose a new version of the RESPLAN planning algorithm based on two significant enhancements. The first incorporates *planning landmarks* [8, 10] into a pruning strategy in the search needed to prove state resilience. This helps avoid unnecessary explorations and yields substantial performance gains, especially when no resilient plan exists. The second introduces a planning adaptation strategy exploiting the notion of *regressed state formulas* to support the search process during (re)planning, reducing the number of iterations required when a resilient plan does exist. We evaluate the new algorithm on several classical planning benchmarks, comparing its performance against the original version and against a compilation of resilient planning into strong FOND planning [6]. Overall the results demonstrate substantial improvements in terms of speed and problem coverage. The remainder of the paper is organized as follows. We begin by formally defining resilient planning and giving an overview of the original RESPLAN algorithm, outlining its main limitations. Then we introduce the new enhancements and the new algorithm incorporating them. Finally, we report on our experimental results and provide conclusions.

## 2 Preliminaries

A classical planning problem is a tuple  $\Pi = \langle F, A, s_0, G \rangle$  where  $F$  is a finite set of facts,  $A$  is finite set of actions,  $s_0 \subseteq F$  is the initial state, and  $G \subseteq F$  is the goal. A state  $s$  is represented as a subset of  $F$ : all facts in  $s$  are true, while facts in  $F \setminus s$  are false by the closed world assumption. Each action  $a \in A$  is represented as a tuple  $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$ , where  $\text{pre}(a) \subseteq F$  are the *preconditions*,  $\text{add}(a) \subseteq F$  are the *positive effects*, and  $\text{del}(a) \subseteq F$  are the *negative effects*. An action  $a$  is applicable in state  $s$  iff  $\text{pre}(a) \subseteq s$ , and we denote the set of actions applicable in state  $s$  with  $A(s)$ . The application of an action  $a \in A(s)$  in  $s$  generates a state  $s[a] = (s \setminus \text{del}(a)) \cup \text{add}(a)$ . A plan  $\pi$  is a sequence of actions in  $A$ , i.e.,  $\pi = (a_1, \dots, a_n)$ . A plan  $\pi = (a_1, \dots, a_n)$  is a solution for a planning problem  $\Pi = \langle F, A, s_0, G \rangle$  iff the induced trajectory of states  $\tau = (s_0, s_1, \dots, s_n)$  is such that for all  $i \in [1, n]$  it holds that  $\text{pre}(a_i) \subseteq s_{i-1}$  and  $G \subseteq s_n$ . Given a plan  $\pi = (a_1, \dots, a_n)$ , we use  $\pi_i$  to denote the action  $a_i$  in  $\pi$ , and  $\tau_i$  to denote  $s_i$  in  $\tau$ .

**Resilient Planning** is a generalization of classical planning that aims to generate  $k$ -resilient plans, that is, plans that can withstand  $k$  action failures during the plan execution [1]. In resilient planning a

\* Corresponding Author. Email: [alfonso.gerevini@unibs.it](mailto:alfonso.gerevini@unibs.it)

failing action does not alter the state of the world, and after failure, such an action cannot be repeated. Resilient solutions can be formalised through the notion of state resilience. Broadly, a  $k$ -resilient state is a state from which an agent can achieve the goal even after  $k$  action failures occur.

**Definition 1** ( $k$ -Resilient State). *Let  $\Pi = \langle F, A, s_0, G \rangle$  be a planning problem,  $s$  a state, and  $k$  a non-negative integer.*

- (i) A goal state  $s \supseteq G$  is  $k$ -resilient in  $\Pi$ .
- (ii) A state  $s \not\supseteq G$  is 0-resilient in  $\Pi$  iff there exists an action  $a \in A(s)$  such that  $s[a]$  is 0-resilient. Alternatively, iff there is a plan from  $s$  that achieves  $G$ .
- (iii) A state  $s \not\supseteq G$  is  $k$ -resilient in  $\Pi$  for  $k \geq 1$  iff there exists an action  $a \in A(s)$  such that (1)  $s[a]$  is  $k$ -resilient in  $\Pi$  and (2)  $s$  is  $(k-1)$ -resilient in  $\langle F, A \setminus \{a\}, s_0, G \rangle$ .

The notion of  $k$ -resilient plan follows naturally from the definition of  $k$ -resilient state. Indeed, for a plan to be able to withstand  $k$  failures, all states traversed by the plan must be  $k$ -resilient.

**Definition 2** ( $k$ -Resilient Plan). *Given a planning problem  $\Pi = \langle F, A, s_0, G \rangle$ , a solution plan for  $\Pi$  that induces a state trajectory  $(s_0, s_1, \dots, s_n)$  is  $k$ -resilient for  $\Pi$  if, for all  $0 \leq i < n$ , it holds that  $s_i$  is  $k$ -resilient in  $\Pi$ .*

A Resilient Planning problem takes as input a Planning problem  $\Pi$  and a user-specified integer  $K$  denoting the number of failures that the plan must be able to withstand.

**Definition 3** (Resilient Planning problem). *A resilient planning problem is a pair  $\langle \Pi, K \rangle$  where  $\Pi$  is a planning problem and  $K$  is an non-negative integer. A solution for  $\langle \Pi, K \rangle$  is a  $K$ -resilient plan for  $\Pi$ .*

This work improves the existing techniques for Resilient Planning by leveraging *planning landmarks* [8, 10], i.e., facts that must be true at some point in any trajectory induced by a solution plan.

**Definition 4** (Landmark). *Given a planning problem  $\Pi = \langle F, A, s_0, G \rangle$ , a fact  $f \in F$  is a landmark if, for every trajectory  $\tau$  induced by a solution plan for  $\Pi$ ,  $\exists i \leq |\tau| : f \in \tau_i$ .*

We denote with  $\text{LANDMARKS}(F, A, s, G)$  a subset of the landmarks starting from state  $s$  through the actions  $A$ . We do not consider facts that are already true in  $s$  to be landmarks.

In the following, we will also exploit the notion of *regressed state formulas*, where a state formula  $\phi$  is a subset of  $F$  denoting a partial state, i.e., any state  $s$  where the conjunction of the facts in  $\phi$  are true (written  $s \models \phi$ ), and every  $f \in F \setminus \phi$  is either true or false.

**Definition 5** (Regression of a State Formula). *Given a state formula  $\phi \subseteq F$  and an action  $a \in A$ , the regression of  $\phi$  over  $a$  is defined as:*

$$\text{Reg}(\phi, a) = (\phi \setminus \text{add}(a)) \cup \text{pre}(a).$$

### 3 RESPLAN: Overview and Limitations

One approach to solving a resilient planning problem  $\langle \Pi, K \rangle$  is to generate candidate plans for  $\Pi$  and verify that every state along these plans is  $K$ -resilient. This requires accounting for both possible outcomes of each action – success and failure – and using a classical planner to iteratively address these contingencies, similar to the iterative replanning algorithms used in fully observable non-deterministic

---

#### Algorithm 1 High-level RESPLAN/RESPLAN 2 algorithm

---

**Input** Resilient Planning problem  $\langle \Pi = \langle F, A, s_0, G \rangle, K \rangle$   
**Output**  $K$ -resilient plan  $\pi^K$  if it exists; unsolvable, otherwise

```

1: if LMNonResilient( $s_0, K, \emptyset$ ) then ▷ RESPLAN 2 excl
2:   return unsolvable
3: Open  $\leftarrow \{ \langle s_0, K, \emptyset \rangle \}$ ;  $\mathcal{R}_\uparrow \leftarrow \emptyset$ ;  $\mathcal{R}_\downarrow \leftarrow \emptyset$ 
4: while Open  $\neq \emptyset$  do
5:    $\langle s, k, V \rangle \leftarrow$  extract last node inserted in Open
6:   if not InClosedList( $s, k, V$ ) then ▷ Mod
7:     if RCheck( $s, k, V$ ) then ▷ Mod
8:       continue
9:      $\pi, \tau \leftarrow$  GenCandidatePlan( $\Pi, s, k, V$ ) ▷ Mod
10:    if  $\pi$  is null then
11:      UpdateNonResilient( $s, k, V$ )
12:    else
13:      ProcessPlan( $\pi, \tau, s, k, V$ ) ▷ Mod
14:  if  $s_0$  is  $K$ -resilient then
15:     $\pi^K \leftarrow$  ExtractSolution( $\Pi, K$ )
16:  return  $\pi^K$ 
17: else
18:  return unsolvable
```

---

(FOND) planning [12, 4, 16, 15]. This is the core idea behind the RESPLAN algorithm for resilient planning [1].

To contextualize the limitations of RESPLAN and motivate our contributions, we briefly describe the algorithm. For clarity, Algorithm 1 presents the high-level pseudocode of RESPLAN and RESPLAN 2, which share the overall same structure, with the exception of lines 1 and 2 – specific to RESPLAN 2.

RESPLAN takes as input a resilient planning problem  $\langle \Pi = \langle F, A, s_0, G \rangle, K \rangle$  and returns a  $K$ -resilient plan if one exists; otherwise, it reports *unsolvable*. The algorithm maintains an Open list of nodes  $\langle s, k, V \rangle$ , where each node represents the subproblem of verifying whether state  $s$  is  $k$ -resilient in the modified task  $\langle F, A \setminus V, s, G \rangle$ , with  $V$  being the set of *forbidden actions* (i.e., the actions hypothesized to have failed their execution when state  $s$  is reached in the plan execution). To solve a node  $\langle s, k, V \rangle$ , the algorithm generates a plan  $\pi$  for  $\langle F, A \setminus V, s, G \rangle$ , and for every state  $s'$  along the trajectory of  $\pi$ , it inserts into Open the corresponding tuples  $\langle s', k, V \rangle$  and  $\langle s', k-1, V \cup a \rangle$  (in accordance with Definition 1), where  $a$  is the action applied at that step. Importantly, Open is organized as a Last-In-First-Out (LIFO) queue. This ensures that the most recently added nodes – those deeper in the plan and with lower resilience levels – are explored first. As a result, when a node  $\langle s, k, V \rangle$  is extracted, the outcomes of both  $\langle s[a], k, V \rangle$  and  $\langle s, k-1, V \cup a \rangle$  are already known. If both were proven to be resilient, then  $\langle s, k, V \rangle$  is also resilient by Definition 1; otherwise, a new plan is generated from  $s$ . States verified to be resilient are added to the set  $\mathcal{R}_\uparrow$ , while those proven non-resilient are added to  $\mathcal{R}_\downarrow$  by storing the corresponding  $\langle s, k, V \rangle$  tuples. Together, these two sets form a closed list, avoiding redundant evaluations.

**Initialization and main loop:** The algorithm begins by inserting the initial node  $\langle s_0, K, \emptyset \rangle$  into Open (line 3). At each iteration of the main loop (line 4), the last-inserted node  $\langle s, k, V \rangle$  is removed from Open (line 5):

- First, in line 6, the algorithm checks whether this node is already solved by calling *InClosedList*( $s, k, V$ ), which simply returns the solution to the membership check  $\langle s, k, V \rangle \in \mathcal{R}_\uparrow \cup \mathcal{R}_\downarrow$ .
- If not, it calls *RCheck*( $s, k, V$ ) in line 7 to determine whether  $s$  can

be shown to be  $k$ -resilient using only the resilient states in  $\mathcal{R}_\uparrow$ . If the check succeeds,  $\langle s, k, V \rangle$  is added to  $\mathcal{R}_\uparrow$ , and the algorithm proceeds to the next iteration.

If  $s$  cannot yet be proven  $k$ -resilient, it indicates that the current candidate plan is not  $k$ -resilient so the algorithm attempts to compute a new candidate plan by calling  $\text{GenCandidatePlan}(\Pi, s, k, V)$  (line 9). This function invokes a planner to find a plan  $\pi$  for  $\langle F, A \setminus V, s, G \rangle$  that avoids any state known to be non- $k$ -resilient state (i.e., those in  $\mathcal{R}_\downarrow$ ), and returns  $\pi$  (or null, if no plan is found) together with its associated state trajectory  $\tau$ :

- If no plan is found (line 10), then all candidate plans for  $s$  have been exhausted, and  $s$  is marked as non-resilient, updating  $\mathcal{R}_\downarrow$  accordingly (line 11).
- If a plan is found, the call to  $\text{ProcessPlan}(\pi, \tau, s, k, V)$  (line 13) generates the new subproblems – nodes that must also be checked for resilience – and adds them to the Open list.

**Termination:** The algorithm terminates when the Open list is empty. At that point, there are two possible cases to consider: the initial node  $\langle s_0, K, \emptyset \rangle$  is either in  $\mathcal{R}_\uparrow$  or in  $\mathcal{R}_\downarrow$ .

- If  $\langle s_0, K, \emptyset \rangle$  is in  $\mathcal{R}_\uparrow$ , then a  $K$ -resilient plan exists (lines 14–16) and is returned as an output through the  $\text{ExtractSolution}$  function;
- if it is in  $\mathcal{R}_\downarrow$ , then the problem is determined to be unsolvable (lines 17–18).

**Limitations:** RESPLAN suffers from two main limitations. First, it can only identify that a state is non-resilient when the call to  $\text{GenCandidatePlan}(\Pi, s, k, V)$  fails to return a plan (line 10). This means that RESPLAN needs to exhaust all possible plans before identifying a state as non-resilient with respect to  $k$  failures, which can be computationally expensive, especially for higher resilience levels. Second, observe that a state  $s$  being  $k$ -resilient implies the existence of a  $k$ -resilient plan starting from  $s$ . This means that we can regard the resilient states verified during RESPLAN’s execution as solved suffixes of a resilient plan. Yet, upon identifying a state as non-resilient, RESPLAN computes an entirely new plan, without trying to reconnect to the already solved suffixes, potentially resulting in redundant computations.

## 4 Enhancing Resilient Planning

This section introduces two novel mechanisms to overcome the limitations of RESPLAN discussed above.

To address the first limitation, we propose an early detection mechanism for non-resilient states based on landmark analysis. Specifically, we establish sufficient conditions linking the resilience level to the number of actions that can achieve a given landmark. As formalized in Section 4.1, this connection enables the identification of non-resilient states without resorting to exhaustive search.

To mitigate the second limitation, we allow the planner to treat previously verified resilient states as disjunctive goals during the replanning performed by  $\text{GenCandidatePlan}$ . This adaptive strategy allows the prefix of a new candidate plan to be connected to already verified plan suffixes, thereby leveraging resilient states found in earlier iterations.

A key challenge in implementing the second enhancement lies in identifying which states can serve as valid disjunctive goals during replanning. Consider the subproblem  $\langle s, k, V \rangle$ , where we aim to determine whether  $s$  is  $k$ -resilient in  $\langle F, A \setminus V, s_0, G \rangle$ . Since we are constructing a  $k$ -resilient plan, we can exploit resilient states

$\langle s', k', V' \rangle \in \mathcal{R}_\uparrow$  such that  $k \leq k'$  and  $V \subseteq V'$ . However, in RESPLAN, the sets  $V$  and  $k$  are tightly coupled –  $V$  expands at the same rate as  $k$  decreases – resulting in the effective set of eligible goal states being constrained to:

$$\mathcal{G}(s, k, V) = \{s' \mid \langle s', k', V' \rangle \in \mathcal{R}_\uparrow \wedge k = k' \wedge V = V'\}.$$

That is, we can only reuse states that are  $k$ -resilient in the *exact* same planning instance  $\langle F, A \setminus V, s_0, G \rangle$ . This restriction drastically limits the effectiveness of our proposal. Moreover, using fully specified states as goals is often impractical, especially in large or complex planning domains. However, it is important to note that demonstrating the  $k$ -resilience of a state  $s$  does not require access to the entire action set  $A \setminus V$ , but only to a subset sufficient for constructing resilient plans across all failure levels up to  $k$ . This observation motivates the introduction of resilience certificates, formalized in Section 4.2, which compactly capture the sufficient conditions under which a state is known to be resilient. These certificates are key to making the adaptive replanning strategy both practical and effective, as they allow the planner to generalize beyond exact matches of  $k$  and  $V$  and reuse previously verified states more broadly.

### 4.1 Connecting Landmarks to State Resilience

In this section, we leverage landmarks alongside a desired resilience level  $k$ , to identify a sufficient condition to label a state as non-resilient. The underlying intuition is that, since up to  $k$  action failures may occur, potentially affecting the achievers of a landmark, at least  $k+1$  actions must exist that achieve the same landmark for a resilient plan to exist. If such redundancy is not found, the state can be deemed non-resilient. Formally, let  $\text{Ach}(f, A) = \{a \in A \mid f \in \text{add}(a)\}$  be the subset of actions in  $A$  that achieve fact  $f$ . We show that the existence of a landmark  $f$  such that  $|\text{Ach}(f, A)| \leq k$  is *sufficient* to say that  $s$  cannot be  $k$ -resilient.

**Theorem 1.** *Let  $\Pi = \langle F, A, s, G \rangle$  be a planning problem and  $k$  a non-negative integer. If there is a fact-landmark  $f \in \text{LANDMARKS}(F, A, s, G)$  such that  $|\text{Ach}(f, A)| \leq k$ , then  $s$  is **not**  $k$ -resilient in  $\Pi$ .*

*Proof.* Assume that there exists a landmark  $f \in \text{LANDMARKS}(F, A, s, G)$  such that  $|\text{Ach}(f, A)| \leq k$  and  $s$  is  $k$ -resilient. We proceed by contradiction, showing that this cannot be for any of the cases in Definition 1.

Case (i): if  $s$  is a goal state, then  $\text{LANDMARKS}(F, A, s, G) = \emptyset$  and there cannot exist the landmark  $f$ .

Case (ii): if  $k = 0$ , we have that  $|\text{Ach}(f, A)| \leq 0$ , i.e., no action procures  $f$ . Therefore, no solution plan for  $\langle F, A, s, G \rangle$  exists.

Case (iii): if  $k > 0$ , there must exist an action  $a \in A(s)$  such that  $s[a]$  is  $k$ -resilient and  $s$  is  $(k-1)$ -resilient even without  $a$  available. Hence, decreasing  $k$  by 1 consumes one action and, since  $f$  is a landmark, the consumed action will necessarily be an achiever at some point. If  $|\text{Ach}(f, A)| \leq k$ , by the time we reach 0-resilience,  $\text{Ach}(f, A)$  will be depleted and  $f$  will no longer be achievable, making the problem unsolvable.  $\square$

### 4.2 Resilience Certificates

We present a new notion, *resilience certificate*, which represents sufficient conditions to label a state as resilient. A  $k$ -resilience certificate for a problem  $\Pi = \langle F, A, s_0, G \rangle$  is a pair  $\langle \phi, D \rangle$  where  $\phi \subseteq F$  is a state formula and  $D \subseteq A$  is a set of actions. The certificate  $\langle \phi, D \rangle$

carries the meaning that any state  $s$  that satisfies  $\phi$  can be proven to be  $k$ -resilient using the actions in  $D$ .

**Definition 6** ( $k$ -Resilience Certificate). Let  $\Pi = \langle F, A, s_0, G \rangle$  be a planning problem, and  $k$  a non-negative integer.

- (i)  $\langle G, \emptyset \rangle$  is a  $k$ -resilience certificate.
- (ii) If  $\langle \phi, D \rangle$  is a 0-resilience certificate, then  $\langle \text{Reg}(\phi, a), D \cup \{a\} \rangle$  is a 0-resilience certificate, for any  $a \in A$ .
- (iii) If  $\langle \phi, D \rangle$  is a  $k$ -resilience certificate and  $\langle \phi', D' \rangle$  is a  $(k-1)$ -resilience certificate, then  $\langle \text{Reg}(\phi, a) \cup \phi', D \cup D' \cup \{a\} \rangle$  is a  $k$ -resilience certificate, for any  $a \in A$ .

**Theorem 2.** Let  $\Pi = \langle F, A, s, G \rangle$  be a planning problem, and  $\langle \phi, D \rangle$  be a  $k$ -resilience certificate for  $\Pi$ . Any state  $s \in S$  such that  $s \models \phi$  is  $k$ -resilient in  $\Pi$ .

*Proof (sketch).* It can be shown by construction that the theorem holds for the three cases of Definition 6. For case (i),  $s \supseteq G$  is a goal state and, therefore,  $k$ -resilient by Definition 1. For the 0-resilience certificates of case (ii),  $\phi$  is simply the regression of a plan while  $D$  are the actions in the plan. Since a state is 0-resilient iff a plan to the goal exists, this case holds, too. For the general case (iii),  $\phi$  represents the conjunction of the regression of  $G$  over a  $k$ -resilient plan and the alternative  $(k-1)$ -resilient plan, so both plans will be executable in any state  $s \models \phi$ .  $\square$

Resilience certificates generalize resilient states, as a single certificate captures a set of resilient states. This enables a more practical implementation of the adaptation strategy outlined above, since instead of associating a state solely with the resilience level and a planning problem, we can record the actual set of actions that were used to prove the state resilient and the formula that the state needs to satisfy to execute these actions.

### 4.3 The RESPLAN 2 Algorithm

RESPLAN 2 is an enhanced version of the RESPLAN algorithm, designed to overcome the limitations discussed in Section 3 by incorporating two key features: (1) an early detection mechanism for non-resilient states, and (2) an adaptive replanning strategy. While RESPLAN 2 preserves the original high-level structure of the algorithm (see Algorithm 1), it introduces several modifications to support these new capabilities. The pseudocode for the most significantly affected functions is presented in Algorithm 2.

The introduction of resilience certificates brings fundamental changes to the algorithm, as  $\mathcal{R}_\uparrow$  no longer stores resilient states but instead stores resilience certificates. In RESPLAN,  $\mathcal{R}_\uparrow$  was populated with nodes  $\langle s, k, V \rangle$  whenever a state  $s$  was proven to be  $k$ -resilient in  $\langle F, A \setminus V, s_0, G \rangle$ . In contrast, RESPLAN 2 stores  $k$ -resilience certificates in the form of triples  $\langle \phi, k, D \rangle$ , denoting that  $\langle \phi, D \rangle$  is a  $k$ -resilience certificate. While the *Open* list still operates on nodes of the form  $\langle s, k, V \rangle$ , these nodes must now be evaluated against the certificates in  $\mathcal{R}_\uparrow$ . We formalize this by writing  $\langle s, k, V \rangle \sim \langle \phi, k', D \rangle$  to mean that  $s \models \phi$ ,  $k \leq k'$ , and  $D \subseteq A \setminus V$ . That is, the certificate  $\langle \phi, D \rangle$  guarantees that  $s$  is  $k$ -resilient in the problem  $\langle F, A \setminus V, s, G \rangle$ .

The adoption of resilience certificates thus impacts all functions that read from or update  $\mathcal{R}_\uparrow$ :

- *InClosedList*( $s, k, V$ ) now returns true if either  $\langle s, k, V \rangle \in \mathcal{R}_\downarrow$  or there exists a compatible (under the  $\sim$  relation) certificate in  $\mathcal{R}_\uparrow$ .

#### Algorithm 2 New and modified RESPLAN 2 functions

---

```

1: function RCheck( $s, k, V$ )
2:   for  $a \in A(s) \setminus V$  do
3:     for  $\langle \phi', k', D' \rangle, \langle \phi'', k'', D'' \rangle \in \mathcal{R}_\uparrow$  do
4:       if  $\langle s[a], k, V \rangle \sim \langle \phi', k', D' \rangle \wedge \langle s, k-1, V \cup \{a\} \rangle \sim$ 
          $\langle \phi'', k'', D'' \rangle$  then
5:          $\phi = \text{Reg}(\phi', a) \cup \phi''$ 
6:          $D = D' \cup D'' \cup \{a\}$ 
7:         add  $\langle \phi, k, D \rangle$  to  $\mathcal{R}_\uparrow$ 
8:         return True
9:   return False
10:
11: function LMNonResilient( $s, k, V$ )
12:   for  $f \in \text{LANDMARKS}(F, A, s, G)$  do
13:     if  $|\text{Ach}(f, A \setminus V)| \leq k$  then
14:       return True
15:   return False
16:
17: procedure ProcessPlan( $\pi, \tau, s, k, V$ )
18:   if  $k > 0$  then
19:     for  $i = 1$  to  $i = |\pi|$  do
20:       if LMNonResilient( $s, k, V$ ) then
21:         break
22:       else
23:         insert  $\langle \tau_{i-1}, k, V \rangle$  in Open
24:         insert  $\langle \tau_{i-1}, k-1, V \cup \{\pi_i\} \rangle$  in Open
25:   else
26:      $\phi \leftarrow G, D \leftarrow \emptyset$ 
27:     for  $i = |\pi|$  to  $i = 1$  do
28:        $\phi \leftarrow \text{Reg}(\phi, \pi_i), D \leftarrow D \cup \{\pi_i\}$ 
29:       add  $\langle \phi, 0, D \rangle$  to  $\mathcal{R}_\uparrow$ 

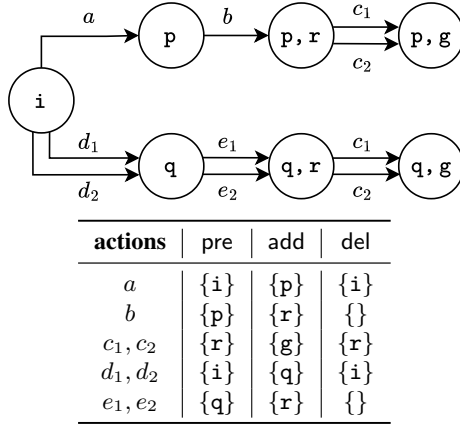
```

---

- *RCheck*( $s, k, V$ ), which determines whether  $s$  can be proven  $k$ -resilient based on known information, has been extended to reason with certificates (see Algorithm 2). It searches for two certificates  $\langle \phi', k', D' \rangle$  and  $\langle \phi'', k'', D'' \rangle$  in  $\mathcal{R}_\uparrow$  (line 3) that together support the construction of a new certificate compatible with  $\langle s, k, V \rangle$  (line 4). If such a pair is found, a new certificate is built according to Definition 6 and added to  $\mathcal{R}_\uparrow$  (lines 5–7).
- *ProcessPlan*( $\pi, \tau, s, k, V$ ) originally added 0-resilient states directly to  $\mathcal{R}_\uparrow$  whenever  $k = 0$  and a solution plan was found. In its updated version, it instead adds 0-resilience certificates to  $\mathcal{R}_\uparrow$  by regressing  $G$  over  $\pi$  (lines 26–29).

#### 4.3.1 Early Detection of Non-Resilient States through Landmarks

RESPLAN 2 introduces a new function, *LMNonResilient*( $s, k, V$ ), which returns True if there exists a landmark  $f$  such that  $|\text{Ach}(f, A \setminus V)| \leq k$  (see lines 11–15 of Algorithm 2). In other words, it detects whether the state  $s$  can be deemed non-resilient based on the sufficient conditions of Theorem 1. When the function returns True, the algorithm immediately labels  $s$  as non-resilient. This check is performed before inserting any node into the *Open* list. This includes two instances: (1) during initialization (lines 1 and 2 of Algorithm 1), and (2) during the generation of new subproblems in the call to *ProcessPlan*( $\pi, \tau, s, k, V$ ) (lines 20 and 21 of Algorithm 2). To illustrate the advantage of this new feature, consider a resilient planning problem  $\langle \Pi = \langle F, A, s_0, G \rangle, 2 \rangle$ , where  $F = \{p\}$ ,  $A = \{a, b\}$  with both actions having no preconditions



**Figure 1.** Example illustrating RESPLAN 2's adaptive replanning strategy.

and both achieving  $p$ ,  $s_0 = \emptyset$ , and  $G = \{p\}$ . In this case, the goal  $p$  is the only landmark. Since  $|Ach(p, A)| = 2$ , RESPLAN 2 immediately detects that  $s_0$  is non-resilient and returns *unsolvable* without entering the main search loop. In contrast, RESPLAN would require several iterations and planner calls to reach the same conclusion. In more complex planning tasks – especially those with higher resilience levels – this early detection mechanism leads to significant performance gains, as demonstrated in our experimental evaluation.

#### 4.3.2 Adaptive Replanning with Resilience Certificates

As anticipated at the start of Section 4, the adaptive strategy is realized by targeting known resilient states as disjunctive goals during replanning episodes. This is implemented through a modification of the *GenCandidatePlan*( $\Pi, s, k, V$ ) function, which, we recall, invokes a planner to solve the planning problem  $\langle F, A \setminus V, s_0, G \rangle$  while avoiding states known to be non- $k$ -resilient, as recorded in  $\mathcal{R}_\downarrow$ <sup>1</sup>.

RESPLAN2 further modifies the planner to monitor whether any state  $s'$  encountered during the search is compatible with a  $k$ -resilience certificate in  $\mathcal{R}_\uparrow$ . If such a state is found, the search terminates early, returning a plan that reaches  $s'$ . By Theorem 2, this plan effectively reconnects to a previously solved resilient suffix starting from  $s'$ . Conceptually, this is equivalent to solving the planning task  $\langle F, A \setminus V, s_0, \mathcal{G}(s, k, V) \rangle$  where

$$\mathcal{G}(s, k, V) = \{\phi \mid \langle \phi, k', D \rangle \in \mathcal{R}_\uparrow \wedge k \leq k' \wedge V \cap D = \emptyset\}$$

are a set of disjunctive goals eligible for node  $\langle s, k, V \rangle$  from the resilient set  $\mathcal{R}_\uparrow$ .

Notably, resilience certificates are generalizable across subproblems. For instance, we may have both  $\langle s, k, V \rangle \sim \langle \phi, k', D \rangle$  and  $\langle s', k, V' \rangle \sim \langle \phi, k', D \rangle$ , even when  $V \neq V'$ . Consequently, while solving  $\langle s, k, V \rangle$ , the planner can exploit certificates discovered in unrelated subproblems. This greatly enhances reuse of previously acquired information and improves the efficiency of the overall search process.

We illustrate the proposed adaptive replanning strategy through an example. Consider a resilient planning problem  $\langle \Pi = \langle F, A, s_0, G \rangle, 1 \rangle$  ( $K = 1$ ) with facts  $F = \{i, p, q, r, g\}$ , actions  $A = \{a, b, c_1, c_2, d_1, d_2, e_1, e_2\}$ , initial state  $s_0 = \{i\}$ , and goal condition  $G = \{g\}$ . The precondition, add and delete lists of each action, and the induced state-space are shown in Figure 1.

Suppose that, in the first iteration, RESPLAN 2 generates the candidate plan  $(a, b, c_1)$ . The algorithm will prove that state  $s = \{p, r\}$  is 1-resilient by exploiting the alternative action  $c_2$  to recover from potential failures. Consequently, RESPLAN 2 records the resilience certificate  $\langle \{r\}, \{c_1, c_2\} \rangle$ , indicating that any state containing  $r$  is 1-resilient as long as actions  $c_1$  and  $c_2$  remain available.

Continuing the process, state  $\{p\}$  is eventually found to be non-1-resilient, prompting the algorithm to backtrack to the initial state  $\{i\}$  and initiate a new replanning episode. Assume that this time, *GenCandidatePlan*( $\Pi, s, k, V$ ) returns plan  $(d_1, e_1)$ , which reaches state  $\{q, r\}$ . Because this state satisfies the conditions of the previously recorded resilience certificate, the planner halts search early, and returns the partial plan. As a result, plan prefix  $(d_1, e_1)$  is successfully connected to the verified suffix  $(c_1)$ , and, eventually, RESPLAN 2 will yield the complete 1-resilient plan  $(d_1, e_1, c_1)$ .

Crucially, this kind of plan reuse would not be feasible without the introduction of resilience certificates. While the original algorithm, RESPLAN, might also have established that  $\{p, r\}$  is 1-resilient, it would have required independently verifying the resilience of  $\{q, r\}$ , leading to unnecessary additional computation.

The structure illustrated in this example is commonly found in “crafting domains”, where a number of *ingredients* must be gathered to manufacture the goal product. Similarly, delivery domains benefit from this adaptive replanning strategy, particularly when certain segments of the transportation network exhibit resilience. For example, in a logistics scenario, moving packages within a city might be  $k$ -resilient, while intercity transfers are more fragile – making it advantageous to reuse intra-city transportation plans where possible.

#### 4.3.3 Theoretical Guarantees

The new features introduced in RESPLAN 2 do not alter the theoretical guarantees of the original algorithm. RESPLAN 2 is sound, complete, and guaranteed to terminate in a finite number of iterations, provided the underlying classical planner used in *GenCandidatePlan* possesses these same properties.

**Theorem 3.** RESPLAN 2 is sound, complete, and terminates after a finite number of iterations.

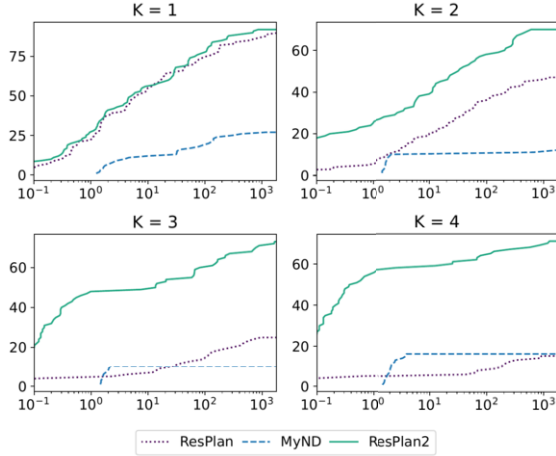
*Proof (sketch).* The early detection mechanism exploiting landmarks only labels a state as non-resilient when the sufficient condition in Theorem 1 is satisfied. Since this condition is conservative, no resilient state will be incorrectly labeled as non-resilient, thus preserving completeness.

The adaptive replanning strategy connects the current candidate plan prefix to a previously verified resilient suffix. By Theorem 2, this connection is valid and does not compromise soundness. Importantly, the remaining plan *prefix* is still subject to verification by the algorithm, ensuring correctness.

Additionally, the goal state  $G$  is always included in the disjunctive goal set  $\mathcal{G}(s, k, V)$  of any subproblem  $\langle s, k, V \rangle$  via the default  $K$ -resilience certificate  $\langle G, \emptyset \rangle$ , which serves as the base case. This guarantees that an entirely new plan to the goal remains a valid outcome, preserving completeness.

Hence, since RESPLAN is sound, complete and terminates in a finite number of iterations [1], and the algorithm revisions introduced in RESPLAN 2 do not affect these properties, we have that they also hold for RESPLAN 2.  $\square$

<sup>1</sup> This can be obtained by appropriately modifying the successor state function implemented in the planner to prune the relevant states in  $\mathcal{R}_\downarrow$ .



**Figure 2.** Cumulative coverage (y-axis) vs runtime (x-axis) for increasing values of  $K$ .

## 5 Experimental Analysis

Our implementation of RESPLAN2 builds upon the RESPLAN system, and both adopt FastDownward [7] as the underlying classical planner, running with GBFS and the  $h_{FF}$  heuristic [9], and we compute fact-landmarks using the backchaining method over the Relaxed Planning Graph proposed by Zhu and Givan [17]. The source code of the new system and all the benchmarks that we use in its evaluation are publicly available on GitHub at: <https://github.com/ResilientPlan/ResPlan2>.

In this section, first we experimentally compare RESPLAN2 against two baselines: the original RESPLAN algorithm and a compilation-based approach proposed in [1]. The latter compiles resilient planning problems into fully observable non-deterministic (FOND) planning problems, and uses MYND [14], a state-of-the-art FOND planner capable of computing strong non-cyclic policies. Then, we present an ablation study in which we evaluate the impact of the new features. We expect the adaptive planning search to be particularly effective for solvable resilient planning problems, and the landmarks-based technique to be more useful for proving unsolvability. As benchmarks domains and problems, we use the same benchmark suite used in [1]. All experiments are conducted on a Xeon Gold 6140M CPU running at 2.3 GHz. The timeout for each instance is set to 1800 seconds, and the memory limit is 32 GB.

### 5.1 Problem Coverage and Runtime Evaluation

Here we compare RESPLAN2 with the two baselines MYND and RESPLAN. The results are shown in Figure 2, which plots cumulative coverage (y-axis) against CPU time in seconds (x-axis, log scale) for different resilience levels from  $K = 1$  to  $K = 4$ . Coverage includes both solvable and unsolvable instances. Each plot corresponds to a different value of  $K$ , and the three systems are distinguished by color and line style: MYND (dashed blue), RESPLAN (dotted purple), and RESPLAN2 (solid green). Regarding the comparison among RESPLAN2 and the two baselines, we observe that RESPLAN2 consistently outperforms both RESPLAN and MYND across all resilience levels (especially for  $K > 1$ ). As observed in the experimental evaluation of [1], the limitations of RESPLAN become evident as the resilience level increases. Thanks to the introduction of landmark-based pruning, RESPLAN2 now significantly improves performance, especially in the unsolvable instances where

the required resilience level is high. Another important observation is that RESPLAN2 is able to maintain strong coverage even at lower resilience levels. Overall, these results demonstrate that the combination of adaptive search and landmark pruning in RESPLAN2 is very effective.

Figure 3 shows an instance-wise CPU time comparison of RESPLAN2 against RESPLAN and MYND across varying resilience levels  $K$ . We distinguish between solvable and unsolvable instances to analyze performance across both categories. RESPLAN2 consistently outperforms MYND at all resilience levels, demonstrating superior scalability and computational efficiency.<sup>2</sup> Compared to RESPLAN, performance at  $K = 1$  is largely comparable, with RESPLAN2 showing a slight advantage on harder instances. However, as the resilience level increases ( $K > 1$ ), RESPLAN2 significantly outperforms RESPLAN, particularly on unsolvable instances where RESPLAN frequently exceeds the time limit. Interestingly, at intermediate values of  $K$ , especially  $K = 2$  and  $K = 3$ , a small number of instances are solved exclusively by RESPLAN. This can be attributed to the overhead incurred by RESPLAN2 in managing the additional data structures required for adaptive replanning, which may hinder performance in some specific configurations.

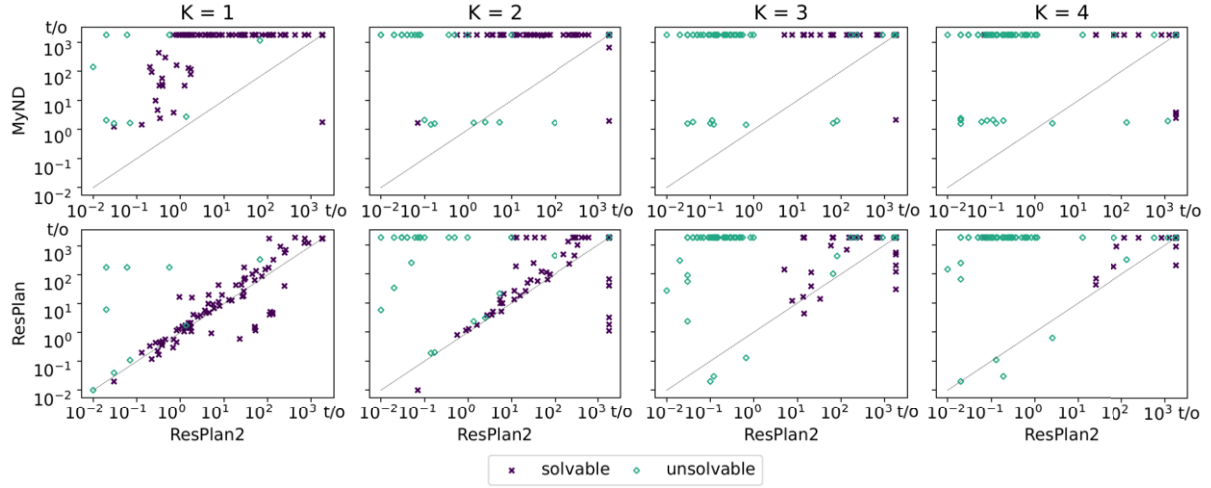
### 5.2 Ablation Study

In Table 1, we present a comprehensive coverage comparison between the baseline RESPLAN and all configurations of the enhanced RESPLAN2 system: RESPLAN with landmark-based pruning only (LRP), RESPLAN with adaptive replanning only (ARP), and the full RESPLAN2 implementation combining both features (RP2). As before, we distinguish between solvable (S) and unsolvable (U) instances and evaluate up to resilience level  $K = 4$ . For the baseline RESPLAN, we report absolute coverage values, while for the other configurations we report only the difference relative to RESPLAN. The results validate our initial expectations: landmark-based pruning yields significant improvements on unsolvable instances. This benefit becomes more pronounced as  $K$  increases, due to the growing number of unsolvable instances and the high cost of exhaustively proving their unsolvability. For solvable instances, the adaptive replanning strategy contributes a modest but consistent improvement in solvable instances, generally outperforms RESPLAN across most settings. The only exceptions are a few Logistics and Rocket instances, which were specifically handcrafted in the original work to accommodate resilient plans. In these cases, the need for replanning is reduced, and the additional overhead of the adaptive strategy may not yield a net benefit.

## 6 Related Work

The work related to RESPLAN2 is similar to the related work that is (more extensively) discussed in [1] for RESPLAN, and is briefly reported in this section. Resilient Planning (RP for short) can be formulated as a specific restriction of Fully-Observable-Non-Deterministic (FOND) planning, which in its general formulation is an EXPTIME-complete problem [13]. Such restriction is similar to the Fault-Tolerant (FT) Planning problem studied in [2, 11], which has been shown to be PSPACE-complete for some restricted fragments [2].

<sup>2</sup> MYND's curves for  $K > 1$  stop to increase after one or few seconds because no or very few additional problems are solved with more time. In other words, MYND saturates coverage after a couple of seconds.



**Figure 3.** RESPLAN2 (x-axis) vs MyND and RESPLAN (y-axis). Points represent runtime in seconds over all instances.

**Table 1.** Coverage difference with respect to the baseline RESPLAN (RP) for Landmark enhancement (LRP), Adaptive Planning enhancement (APRP) and RESPLAN2 (RP2)

| Domain                 | Sol | $K=1$ |     |      |     | $K=2$ |     |      |     | $K=3$ |     |      |     | $K=4$ |     |      |     |
|------------------------|-----|-------|-----|------|-----|-------|-----|------|-----|-------|-----|------|-----|-------|-----|------|-----|
|                        |     | RP    | LRP | APRP | RP2 | RP    | LRP | APRP | RP2 | RP    | LRP | APRP | RP2 | RP    | LRP | APRP | RP2 |
| Driverlog (#20)        | S   | 14    | 0   | 0    | 0   | 1     | 0   | 1    | 1   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   |
|                        | U   | 0     | 0   | 0    | 0   | 0     | 4   | 5    | 5   | 0     | 13  | 1    | 14  | 0     | 15  | 1    | 15  |
| Satellite (#36)        | S   | 15    | 0   | 0    | 0   | 4     | 0   | 2    | 2   | 1     | 0   | 2    | 2   | 0     | 0   | 0    | 0   |
|                        | U   | 1     | 0   | 2    | 3   | 1     | 9   | 1    | 9   | 1     | 13  | 1    | 14  | 1     | 17  | 2    | 17  |
| Storage (#30)          | S   | 10    | 0   | 0    | 0   | 3     | 0   | 2    | 2   | 0     | 0   | 1    | 1   | 0     | 0   | 0    | 0   |
|                        | U   | 5     | 0   | 0    | 0   | 7     | 1   | 0    | 1   | 6     | 2   | 3    | 4   | 5     | 3   | 2    | 4   |
| Zenotravel (#20)       | S   | 11    | 0   | 3    | 3   | 2     | 0   | 3    | 3   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   |
|                        | U   | 1     | 0   | 0    | 0   | 1     | 5   | 2    | 5   | 2     | 11  | 1    | 11  | 2     | 11  | -1   | 11  |
| BlocksworldMA (#10)    | S   | 10    | 0   | 0    | 0   | 10    | 0   | 0    | 0   | 4     | 0   | 2    | 2   | 3     | 0   | 3    | 3   |
|                        | U   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   | 0     | 4   | 2    | 4   | 0     | 4   | 3    | 4   |
| Logistics (#10)        | S   | 10    | 0   | -1   | -1  | 9     | 0   | -3   | -3  | 7     | 0   | -5   | -5  | 3     | 0   | -2   | -2  |
|                        | U   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   |
| Rocket (#10)           | S   | 10    | 0   | 0    | 0   | 9     | 0   | -2   | -2  | 3     | 0   | 2    | 2   | 0     | 0   | 1    | 1   |
|                        | U   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   | 0     | 0   | 0    | 0   | 0     | 4   | 1    | 4   |
| <b>Subtotal</b> (#136) | S   | 80    | 0   | 2    | 2   | 38    | 0   | 3    | 3   | 15    | 0   | 2    | 2   | 6     | 0   | 2    | 2   |
|                        | U   | 7     | 0   | 3    | 3   | 9     | 19  | 8    | 20  | 9     | 43  | 8    | 47  | 8     | 54  | 8    | 55  |
| <b>Total Coverage</b>  | C   | 87    | 0   | 92   | 92  | 47    | 66  | 58   | 70  | 24    | 67  | 34   | 73  | 14    | 68  | 24   | 71  |

As shown in [1], RP can be cast as a particular case of FT planning. However, the RESPLAN algorithm and its enhancements presented in this paper significantly differ from the existing algorithms for FOND and FT planning. Moreover, RP systems can be fairly compared only with FOND planning systems supporting strong *non-cyclic* solutions (as MyND [14]), because in the RP setting there is no fairness assumption. Like PRP [16] and PR2 [15], state-of-the-art FOND planners, RESPLAN2 adopts an iterative replanning approach. These planners likewise exploit regression to compactly represent policies and guide replanning, but in contrast to RESPLAN2, their aim is to quickly close the policy by targeting states already addressed, rather than to reuse previously discovered knowledge. Differently from the FT planning algorithm in [11], RESPLAN2 have no restrictions on the input number of allowed failures, and exploits classical planners almost off-the-shelf, leveraging informed heuristics from classical planning.

## 7 Conclusions

The generation of plans that are robust to execution failures is crucial for many real-world applications. In this work, we have addressed this challenge in the context of classical planning by introducing RESPLAN2, an enhanced variant of the RESPLAN algo-

rithm. RESPLAN2 preserves all the formal properties of RESPLAN while achieving significantly better performance in terms of coverage compared to both the original algorithm and a compilation into strong FOND planning solved by the MyND planner. This holds also in terms of runtime, especially for resilience levels higher than 1. By integrating landmark-based pruning and adaptive planning techniques, RESPLAN2 is able to prove unsolvability more efficiently as the number of tolerated action failures increases, and it also improves the ability to prove solvability in certain instances, thus increasing the resilience of the search. As future work, we intend to explore alternative notions of resilient states and develop additional methods for modeling and handling execution failures. In addition, we plan to test alternative techniques for landmark generation, and to incorporate action landmarks alongside fact landmarks. We will also generalize the problem setting to differentiate between actions that are prone to failure and actions that are guaranteed to succeed. This extension would be highly relevant to practical planning scenarios, allowing a focused treatment of critical actions that impact resilience. Another direction of future work is studying how resilient plans affect plan quality, and developing methods to generate resilient plans taking plan quality metrics into account.

## Acknowledgments

This work has been supported by: MUR PRIN-2020 project RIPER (n. 20203FFYLK); PNRR MUR project PE0000013-FAIR, cascade funding call, ResilientPlans; PNRR MUR project SERICS (PE00000014), cascade funding call, SOS-AI; AI4WATER project is part of the PRIMA Programme supported by the European Union and by the Italian Ministry of University and Research.

## References

- [1] D. Aineto, A. Gaudenzi, A. Gerevini, A. Rovetta, E. Scala, and I. Serina. Action-failure resilient planning. In *Proceedings of the European Conference on Artificial Intelligence (ECAI)*, pages 44–51. IOS Press, 2023.
- [2] C. Domshlak. Fault tolerant planning: Complexity and compilation. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 23, pages 64–72, 2013.
- [3] R. E. Fikes and N. J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [4] J. Fu, V. Ng, F. B. Bastani, and I. Yen. Simple and fast strong cyclic planning for fully-observable nondeterministic planning problems. In *IJCAI*, pages 1949–1954, 2011.
- [5] H. Geffner and B. Bonet. *A Concise Introduction to Models and Methods for Automated Planning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2013.
- [6] M. Ghallab, D. S. Nau, and P. Traverso. *Automated Planning and Acting*. Cambridge University Press, 2016.
- [7] M. Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [8] M. Helmert and C. Domshlak. Landmarks, critical paths and abstractions: What’s the difference anyway? In *Proceedings of the 19th International Conference on Automated Planning and Scheduling (ICAPS)*, 2009.
- [9] J. Hoffmann and B. Nebel. The ff planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research*, 14: 253–302, 2001.
- [10] J. Hoffmann, J. Porteous, and L. Sebastia. Ordered landmarks in planning. In *Proceedings of the 14th International Conference on Automated Planning and Scheduling (ICAPS)*, 2004.
- [11] R. M. Jensen, M. M. Veloso, and R. E. Bryant. Fault tolerant planning: Toward probabilistic uncertainty models in symbolic non-deterministic planning. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pages 335–344, 2004.
- [12] U. Kuter, D. Nau, E. Reisner, and R. P. Goldman. Using classical planners to solve nondeterministic planning problems. In *Proceedings of the Seventeenth International Conference on Automated Planning and Scheduling (ICAPS)*, pages 190–197, 2008.
- [13] M. L. Littman. Probabilistic propositional planning: Representations and complexity. In *AAAI/IAAI*, pages 748–754. AAAI Press / The MIT Press, 1997.
- [14] R. Matz Müller, M. Ortlieb, M. Helmert, and P. Bercher. Pattern database heuristics for fully observable nondeterministic planning. In *ICAPS*, pages 105–112. AAAI, 2010.
- [15] C. Muise, S. McIlraith, and J. Beck. Prp rebooted: Advancing the state of the art in fond planning. 38:20212–20221, 03 2024. doi: 10.1609/aaai.v38i18.30001.
- [16] C. J. Muise, S. A. McIlraith, and J. C. Beck. Improved non-deterministic planning by exploiting state relevance. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. AAAI, 2012.
- [17] L. Zhu and R. Givan. Landmark extraction via planning graph propagation. *ICAPS Doctoral Consortium*, pages 156–160, 2003.